GRU (Gated Recurrent Unit)

Concepts, equations, comparison, applications and PyTorch example

1. What is GRU?

A Gated Recurrent Unit (GRU) is a type of recurrent neural network (RNN) designed to learn patterns in sequential data. It is commonly used for time series, text, speech, and other sequence-processing tasks. GRU uses gates to control how much information is kept from the past and how much new information is added.

2. Why was GRU introduced?

Traditional RNNs can struggle to learn long-term dependencies because gradients may vanish during training. GRU addresses this problem with gating mechanisms that help information flow across time steps. GRU is similar in purpose to an LSTM but has a simpler architecture, usually with fewer gates and no separate cell state.

3. Main components

• Update gate (z_t): controls how much of the previous hidden state is retained versus replaced by new candidate information.

• Reset gate (r_t): controls how much past information is used when creating the candidate hidden state.

• Candidate hidden state (h~_t): proposed new memory based on the current input and gated previous state.

• Hidden state (h_t): the representation passed to the next time step and potentially used for prediction.

4. Core equations

Let x_t be the input at time t, h_(t-1) the previous hidden state, and σ the sigmoid function.

Reset gate:

r_t = σ(W_r x_t + U_r h_(t-1) + b_r)

Update gate:

z_t = σ(W_z x_t + U_z h_(t-1) + b_z)

Candidate hidden state:

h~_t = tanh(W_h x_t + U_h (r_t ■ h_(t-1)) + b_h)

Hidden state update (one common convention):

h_t = (1 - z_t) ■ h_(t-1) + z_t ■ h~_t

Here, ■ means element-wise multiplication. Some libraries use the opposite update-gate convention, so equations may look slightly different while describing the same idea.

5. How GRU works — step by step

1) Read the current input x_t.

2) Compute the reset gate to decide how much previous state to use for candidate memory.

3) Compute the update gate to control the balance between old and candidate information.

4) Create the candidate hidden state with tanh.

5) Combine the old state and candidate state to obtain h_t.

6) Pass h_t to the next time step.

6. GRU vs. vanilla RNN vs. LSTM

Vanilla RNN: Simple and computationally light, but often struggles with long-range dependencies.

GRU: Uses reset and update gates; has a hidden state but no separate cell state; often faster and simpler than LSTM.

LSTM: Uses input, forget, and output gates plus a separate cell state; can be useful when more explicit memory control is beneficial.

7. Advantages

• Can capture longer dependencies than a basic RNN in many tasks.

• Usually has fewer parameters than a comparable LSTM.

• Often trains faster than LSTM, depending on data and implementation.

• Works with variable-length sequences and can be used for classification, forecasting, and sequence generation.

8. Limitations

• It does not guarantee that all long-term dependencies will be learned.

• Performance depends on data quality, model size, and training choices.

• For many language tasks, Transformer architectures may perform better.

• Sequential computation can limit parallelism compared with Transformers.

9. Common applications

• Time-series forecasting, such as demand or sensor readings.

• Sentiment analysis and text classification.

• Speech and audio sequence modeling.

• Activity recognition from sensor data.

• Sequence prediction and anomaly detection.

10. Simple PyTorch example

This example creates a GRU layer and processes a batch of sequences.

```
import torch

import torch.nn as nn

# batch_first=True means input shape: (batch, sequence_length, features)

gru = nn.GRU(

input_size=10,

hidden_size=32,

num_layers=1,

batch_first=True

)
```

x = torch.randn(4, 12, 10) # 4 sequences, 12 time steps, 10 features

output, h_n = gru(x)

print(output.shape) # (4, 12, 32)

print(h_n.shape) # (1, 4, 32)

output contains the hidden representation at every time step. h_n contains the final hidden state for each layer and sequence.

11. GRU in a real project

A typical workflow is:

1) Prepare sequential data and split it into train/validation/test sets.

2) Normalize numeric features if appropriate.

3) Convert data into windows or sequences.

4) Train the GRU with a suitable loss function and optimizer.

5) Evaluate on held-out data and compare against simple baselines.

6) Monitor overfitting and tune sequence length, hidden size, and learning rate.

12. Quick revision

• GRU stands for Gated Recurrent Unit.

• It is a gated form of RNN.

• Its two main gates are reset and update.

• It maintains a hidden state, not a separate LSTM-style cell state.

• It is useful for sequence data and can mitigate vanishing-gradient problems.

• GRU and LSTM are alternatives; neither is always best.

Practice questions

1. Define GRU.

2. What is the role of the reset gate?

3. What does the update gate control?

4. Write the GRU equations and define each symbol.

5. Compare GRU and LSTM.

6. Give two applications of GRU.

7. What do output and h_n represent in PyTorch's nn.GRU?

End of guide.